

NÚCLEO

Última Onda

Game Design Document

I GameCom — X SECOMP-UEM

Autor: Paulo Guilherme Schnauffer

Engine: Unity 6.6 — Universal Render Pipeline 2D

Plataforma: PC (Windows)

Sumário

Ficha Técnica	3
Logline	3
1. Premissa e Lore	3
2. Pilares de Design	5
3. Core Game Loop	6
4. Mecânicas de Jogo	6
5. Direção de Arte	7
6. Áudio	8
7. Feel / Juice	8
8. Decisões Técnicas de Escopo	8
9. Créditos e Licenciamento	8

Execução

1. Clique com o botão direito do mouse no arquivo NÚCLEO_ÚltimaOnda.zip deste mesmo diretório e o baixe.
2. Clique com o botão direito do mouse no arquivo NÚCLEO_ÚltimaOnda.zip, onde tiver sido baixado.
3. Extraia o pacote .zip em uma pasta separada.
4. Clique duas vezes com o botão esquerdo do mouse no arquivo NÚCLEO_ÚltimaOnda.exe, onde tiver sido baixado. **Atenção:** deve estar junto dos outros arquivos.
5. Aproveite!

Requisitos mínimos

- Sistema operacional: Windows 10/11
- 1 GB RAM
- Processador i3 de 5ª geração ou superior
- Nvidia GT 710 ou superior
- 200 MB de armazenamento SSD
- Monitor 16:9

Como jogar

- WASD ou SETAS para se movimentar
- Mouse para interagir com tela de *upgrades*.
- Teclado para digitar seu nome na tela de rankings

Controle recomendado

- Teclado e mouse

Game Design Document (GDD)

Ficha Técnica

Campo	Informação
Título	NÚCLEO: Última Onda
Gênero	Survivors-like (bullet heaven) híbrido com defesa de objetivo
Plataforma	PC (Windows)
Engine	Unity 6.6 — Universal Render Pipeline 2D
Linguagem	C#
Modelo de desenvolvimento	Solo, 40 horas líquidas
Autor	Paulo Guilherme Schnauffer
Contexto	I GameCom — X SECOMP-UEM
Duração de uma run	Até 6 ondas

Logline

Uma estação cujo núcleo computacional começou a falhar. Contenção em colapso. Processos corrompidos convergem para o núcleo a cada ciclo. Você é o último protocolo de defesa ainda ativo — e a única coisa entre esses processos e o apagão total do sistema.

1. Premissa e Lore

1.1 O mundo: Estação Ômega

A ação se passa inteiramente dentro do sistema de defesa da **Estação Ômega**, uma instalação cuja operação depende de um único componente crítico: o **Núcleo**. O que o jogador enxerga na tela — a arena, os inimigos, os projéteis — não é “o mundo real” da estação, é uma **visualização abstrata do próprio sistema imunológico digital do Núcleo em ação**. É por isso que a estética é flat, geométrica e codificada

por cor em vez de figurativa: você não está olhando para corredores da estação, está olhando para dentro da máquina.

A abertura estabelece isso em três frases, deliberadamente secas, quase um log de sistema:

```
"Estação Ômega. Núcleo instável. Contenção em colapso."  
"Processos corrompidos convergem para o núcleo a cada ciclo."  
"Você é o último protocolo de defesa ativo -- não deixe o núcleo cair."
```

Origem da corrupção: os poucos processos de diagnóstico ainda funcionais não chegam a um consenso — a hipótese mais repetida nos logs é terrível: um único bit invertido por radiação cósmica, replicado e amplificado a cada ciclo até virar uma praga de erros. Não houve invasor. O sistema começou a falhar sozinho.

1.2 O Núcleo

O Núcleo é o objetivo que o jogador protege — representado mecanicamente por uma barra de **Integridade** (HP máximo 400). Diferente do jogador (caso adquira o upgrade Redundância), **o Núcleo não se regenera**: qualquer dano sofrido por ele é permanente dentro da run. Cada ponto de integridade perdido é irreversível, o que transforma toda decisão tática numa escolha sobre *quanto do Núcleo você está disposto a sacrificar* para sobreviver mais um pouco.

1.3 Você: o protocolo de defesa

O jogador é a **última instância ativa de um protocolo de vigilância automatizado**, algo entre um antivírus e um processo de emergência que só existe pra intervir quando tudo o mais já falhou. Na linguagem de sistemas reais, é um **watchdog** — uma rotina cujo único trabalho é observar o sistema e agir no instante em que ele para de responder.

Você não defende a estação porque é um herói, mas sim porque **foi criado pra isso**, e enquanto você estiver rodando, o sistema ainda tem uma chance.

1.4 Os inimigos: processos corrompidos

Cada inimigo é o **reskin visual de um erro de sistema real**, e o comportamento em jogo foi desenhado pra ser fiel ao conceito técnico que ele representa.

Inimigo (visual)	Conceito técnico	O que o erro realmente é	Comportamento em jogo
Rusher	Loop Infinito	Um processo que nunca checa sua condição de parada	Avança sem parar direto no alvo mais próximo; rápido, sem desvio
Atirador	DDoS	Ataque que satura a <i>disponibilidade</i> de um serviço	Ignora o jogador de propósito e mira o Núcleo diretamente, à distância
Tanque	Memory Leak	Processo que aloca recursos e nunca os libera, ficando progressivamente mais pesado até comprometer o sistema	Lento e resistente; seu dano de contato cresce com o tempo que ele permanece vivo , até um teto
Boss	Stack Overflow	Erro causado por uma pilha de chamadas que cresce além do limite e estoura	Ciclo de 4 fases: aproxima → telegraph (acúmulo) → overflow (disparo) → cooldown

1.5 Design das ondas

A estação opera em ciclos de processamento, e a cada ciclo, mais instâncias corrompidas convergem pro Núcleo, numa escala crescente que vai da onda 1 até o pico da onda 6. A onda 3 foi calibrada pra ser um marco alcançável pela maioria das runs, funcionando como respiro antes da escalada final; é nas ondas 4, 5 e 6 que a pressão realmente aperta, culminando no Boss.

1.6 Final: vitória e derrota

A run termina ao final da onda 6, com o texto de vitória:

"Onda 6 neutralizada. Núcleo estável. Estação Ômega seguirá operacional."

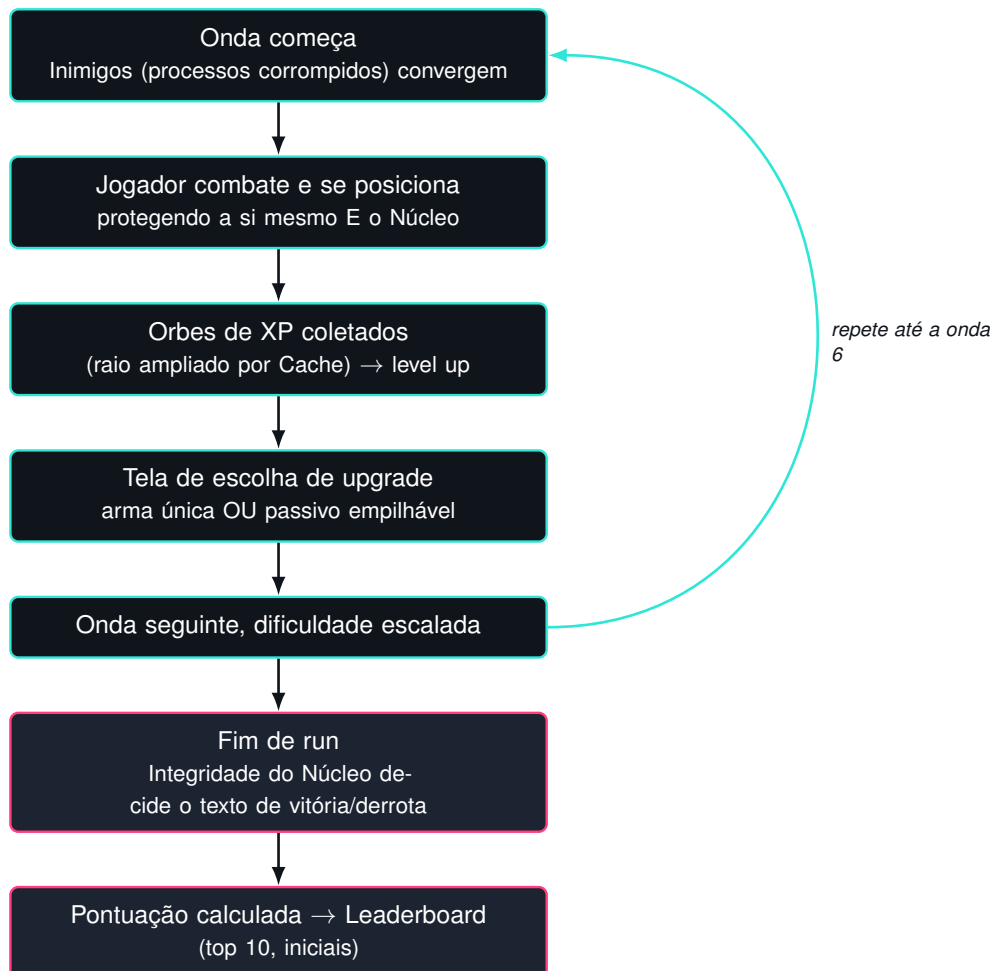
Se o Núcleo chega a 0 antes disso, com o texto de derrota:

*"Núcleo comprometido. Contenção perdida."
"Estação Ômega — fora do ar."*

2. Pilares de Design

1. **Toda mecânica é também uma metáfora técnica.** Não existe elemento decorativo, dano, defesa, inimigo e upgrade sempre significam duas coisas ao mesmo tempo: uma função de jogo e um conceito real de computação.
2. **Pressão dupla.** O jogador defende a si mesmo (HP próprio, recuperável) e um objetivo externo que nunca se regenera (o Núcleo). Isso cria uma tensão que um survivors-like tradicional (só sobreviver) não tem.

3. Core Game Loop



4. Mecânicas de Jogo

4.1 Jogador

- HP máximo: **100**
- Regeneração: só existe com o upgrade passivo **Redundância**
- Progressão por XP e nível (nível real máximo em torno de 12)
- Cada level up abre a tela de escolha de upgrade

4.2 O Núcleo (objetivo protegido)

- Integridade máxima: **400**
- Não regenera sob nenhuma circunstância
- Integridade a 0 = Game Over

4.3 Upgrades

Sete upgrades no total, cada um com nome de conceito de computação e leitura dupla clara:

Nome no jogo	Conceito técnico	Efeito em jogo	Tipo
Daemon	Serviço em segundo plano que roda sozinho, sem intervenção	Torreta automática que atira sozinha	Arma única (inicial)
Fork	Bifurcação: um processo se duplica em múltiplos filhos	Tiro em leque, múltiplos projéteis simultâneos	Arma única
Overclock	Forçar o processador além do clock nominal	Aumenta velocidade de movimento	Passivo (empilha)
Payload	A carga real entregue por um pacote ou exploit	Aumenta dano	Passivo (empilha)
Redundância	Sistemas de backup assumindo quando o principal falha	Regeneração de HP do jogador	Passivo (empilha)
Cache	Memória de acesso rápido para dados usados com frequência	Aumenta raio de coleta de XP	Passivo (empilha)

4.4 Sistema de ondas

Até **6 ondas**, com escalada de dificuldade progressiva. A onda 3 é um marco intermediário alcançável pela maioria das runs; a pressão real se concentra nas ondas finais, culminando no Boss.

4.5 Pontuação e Leaderboard

```
Pontuação = (onda × pontos_por_onda) + (nível × pontos_por_nível)
            + bônus (% HP do Núcleo) + bônus (% HP do jogador)
```

O jogo recompensa primariamente *até onde você chegou*, depois *o quão bem você chegou lá*. O leaderboard (top 10) é exibido com iniciais do jogador.

5. Direção de Arte

- **Assets CC0 (Kenney)**, com curadoria de licenciamento rigorosa.

6. Áudio

- Mixer com 2 grupos sob Master: **Music** e **SFX**.
- Limiter via efeito nativo Compressor no grupo Master.

7. Feel / Juice

- Hit-stop
- Screen shake
- Damage flash

8. Decisões Técnicas de Escopo

Pensado pra um prazo de ~40h líquidas, o projeto prioriza decisões documentadas:

- Input Manager legado, sem setup extra.
- ObjectPool chaveado pelo prefab de origem (não por string), elimina bugs de digitação de chave.

9. Créditos e Licenciamento

- Base de assets majoritariamente CC0 (Kenney).
- "Sci-fi Ambient Drone.wav"
 - Autor: LookIMadeAThing
 - Fonte: <https://freesound.org/people/LookIMadeAThing/sounds/534018/>
 - Licença: Creative Commons CC0 (Domínio Público)
- "8-bit damage sound.wav"
 - Autor: EVRetro
 - Fonte: <https://freesound.org/people/EVRetro/sounds/501104/>

- Licença: Creative Commons CC0 (Domínio Público)
- **"UI Pack - Sci-Fi"**
 - Autor: Kenney Vleugels
 - Fonte: <https://kenney.nl/assets/ui-pack-sci-fi>
 - Licença: Creative Commons CC0 (Domínio Público)
- **"Game Icons"**
 - Autor: Kenney Vleugels
 - Fonte: <https://kenney.nl/assets/game-icons>
 - Licença: Creative Commons CC0 (Domínio Público)
- **"Pixel Shmup"**
 - Autor: Kenney Vleugels
 - Fonte: <https://kenney.nl/assets/pixel-shmup>
 - Licença: Creative Commons CC0 (Domínio Público)